EECS 348 Assignment 5

The code that was given to me:

```
/*
 * Program Name : EECS 348 Assignment 4 - Recursive Object-Or
iented Sudoku Solver
 *
 * Description  : Solves Sudoku puzzles using recursive backt
racking inside a
 *                SudokuSolver class. It finds every solution
for a puzzle, and
 *                prints "No solution found" if there are non
e.
 *
 * Inputs       : puzzle1.txt through puzzle5.txt in the curr
ent folder. Each file
 *                has 9 rows of 9 space-separated symbols: a
digit 1-9 for a given
 *                clue, or an underscore (_) for a blank cel
l.
 *
 * Output       : For each file: the file name, the puzzle as
read, each solution
 *                found (or "No solution found"), and the num
ber of solutions.
 *
 * Collaborators: None.
 *
 * Other sources: - ChatGPT (chatgpt.com): first draft. I use
d its class layout,
 *                  its input checking, and its solution coun
ter.
 *                - Gemini (gemini.google.com): second draft.
```

```
I used its
 *                  cell-by-cell recursion and its single-loo
p block check.
 *                Code marked [MIX] combines ideas from both
drafts. I reviewed
 *                and tested all of it.
 *
 * Author       : Charvi Reddy Konudula
 * Creation date: October 15, 2026
 *
 * Labels in the comments: [CHATGPT] = idea taken from ChatGP
T's draft,
 * [GEMINI] = idea taken from Gemini's draft, [MIX] = combine
d by me from both.
 */
 #include <iostream>   // cout and cerr for printing
#include <fstream>    // ifstream for reading the puzzle file
s
#include <string>     // string for file names and for readin
g symbols

// Constants so the board size is written in one place instea
d of using 9 and 3
// everywhere. ChatGPT's draft hard-coded these numbers, so t
his part is [MIX].
const int SIZE = 9;          // the board is 9 x 9
const int BOX = 3;           // each block is 3 x 3
const char EMPTY = '_';      // symbol for a blank cell
const int NUM_PUZZLES = 5;   // puzzle1.txt through puzzle5.t
xt

// One SudokuSolver object holds one puzzle and solves it.
class SudokuSolver {
private:
    char board[SIZE][SIZE];      // working grid that the sea
rch fills in and erases
```

```
    char original[SIZE][SIZE];   // copy of the puzzle as rea
d, used for printing it
    int solutionCount;           // number of full solutions
found so far

    // [GEMINI] Checks whether 'digit' can go at (row, col).
One loop looks at
    // the row, the column, and the 3x3 block at the same tim
e. boxTop and
    // boxLeft are the top-left corner of the block that hold
s this cell. For
    // each i from 0 to 8, board[row][i] is the row cell and
board[i][col] is
    // the column cell. i / BOX and i % BOX turn i into an of
fset inside the
    // block, so the third check covers all 9 block cells. If
the digit is
    // already in any of them, it returns false. Otherwise it
returns true.
    bool isSafe(int row, int col, char digit) const {
        int boxTop = BOX * (row / BOX);
        int boxLeft = BOX * (col / BOX);
        for (int i = 0; i < SIZE; ++i) {
            if (board[row][i] == digit) {
                return false;
            }
            if (board[i][col] == digit) {
                return false;
            }
            if (board[boxTop + i / BOX][boxLeft + i % BOX] ==
digit) {
                return false;
            }
        }
        return true;
    }
```

```
    // Prints a 9x9 grid, one row per line with spaces betwee
n the symbols.
    // It takes the grid as a parameter so it can print eithe
r the working
    // board (for solutions) or the saved original (for the p
uzzle as read).
    void printBoard(const char grid[SIZE][SIZE]) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                std::cout << grid[r][c];
                if (c < SIZE - 1) {
                    std::cout << ' ';
                }
            }
            std::cout << '\n';
        }
    }

    // [GEMINI] [MIX] The recursive search. 'cell' goes from
0 to 80 and
    // stands for the cell we are working on (row = cell / 9,
col = cell % 9).
    // Gemini's draft moved through the cells in order instea
d of rescanning
    // the grid for an empty cell each time; I turned its ro
w/col pair into a
    // single index.
    //
    // Base case: when cell reaches 81, every cell is filled,
so this is a
    // complete solution. It is counted and printed, and then
the function
    // returns so the search can keep looking for other solut
ions.
    //
```

```
    // If the cell already has a digit (a given clue), skip t
o the next cell.
    //
    // Otherwise try the digits 1 to 9. For each digit that i
sSafe() allows,
    // place it, call search() on the next cell, and then era
se it again.
    // The erase is the backtracking step: it lets the loop t
ry the next
    // digit, and it makes sure the search finds every soluti
on, not just
    // the first one. If no digit fits, the loop ends and the
function
    // returns to the previous cell, which then tries its nex
t digit.
    void search(int cell) {
        if (cell == SIZE * SIZE) {
            ++solutionCount;
            std::cout << "\nSolution " << solutionCount <<
":\n";
            printBoard(board);
            return;
        }
        int row = cell / SIZE;
        int col = cell % SIZE;
        if (board[row][col] != EMPTY) {
            search(cell + 1);
            return;
        }
        for (char digit = '1'; digit <= '9'; ++digit) {
            if (isSafe(row, col, digit)) {
                board[row][col] = digit;
                search(cell + 1);
                board[row][col] = EMPTY;
            }
        }
```

```
    }

public:
    SudokuSolver() : solutionCount(0) {}

    // [CHATGPT] Reads a puzzle file into both grids. I kept
ChatGPT's input
    // checking because Gemini's draft had none. It returns f
alse with an
    // error message if the file will not open, if there are
fewer than 81
    // symbols, or if a symbol is not "_" or a single digit 1
-9. Each symbol
    // is read as a string token (so values like "10" are cau
ght) and its
    // first character is stored in board and in original.
    bool loadPuzzle(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error: cannot open " << filename <<
'\n';
            return false;
        }
        std::string token;
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (!(file >> token)) {
                    std::cerr << "Error: " << filename << " h
as fewer than 81 cells\n";
                    return false;
                }
                bool isBlank = (token == "_");
                bool isDigit = (token.length() == 1 && token
[0] >= '1' && token[0] <= '9');
                if (!isBlank && !isDigit) {
                    std::cerr << "Error: bad symbol '" << tok
```

```
en << "' in " << filename << '\n';
                    return false;
                }
                board[r][c] = token[0];
                original[r][c] = token[0];
            }
        }
        return true;
    }

    // Prints the puzzle exactly as it was read from the fil
e.
    void printOriginal() const {
        printBoard(original);
    }

    // Resets the counter and starts the search at cell 0. Wh
en the search is
    // done, it prints the total number of solutions, or "No
solution found"
    // (the wording the assignment asks for) if the count is
still 0.
    void solve() {
        solutionCount = 0;
        search(0);
        if (solutionCount == 0) {
            std::cout << "No solution found\n";
        } else {
            std::cout << "\nTotal solutions found: " << solut
ionCount << '\n';
        }
    }
};

// main goes through the five puzzle files. For each one it b
uilds the file
```

```
// name, prints a header with the name, creates a new SudokuS
olver, loads the
// puzzle, and only if the load worked prints the puzzle and
solves it.
// A blank line is printed between puzzles.
int main() {
    for (int i = 1; i <= NUM_PUZZLES; ++i) {
        std::string filename = "puzzle" + std::to_string(i) +
".txt";
        std::cout << "=======================================
=\n";
        std::cout << "File: " << filename << '\n';
        std::cout << "=======================================
=\n";
        SudokuSolver solver;
        if (solver.loadPuzzle(filename)) {
            std::cout << "\nPuzzle:\n";
            solver.printOriginal();
            solver.solve();
        }
        std::cout << '\n';
    }
    return 0;
}
```

ChatGPT:

```
#include <iostream>
#include <fstream>
#include <string>

const int SIZE = 9;
const int BOX = 3;
```

```
const char EMPTY = '_';
const int NUM_PUZZLES = 5;

class SudokuSolver {
private:
    char board[SIZE][SIZE];
    char original[SIZE][SIZE];
    int solutionCount;

    // [GEMINI] Check row, column, and 3x3 block.
    bool isSafe(int row, int col, char digit) const {
        int boxTop = BOX * (row / BOX);
        int boxLeft = BOX * (col / BOX);

        for (int i = 0; i < SIZE; ++i) {
            if (board[row][i] == digit) {
                return false;
            }
            if (board[i][col] == digit) {
                return false;
            }
            if (board[boxTop + i / BOX][boxLeft + i % BOX] ==
digit) {
                return false;
            }
        }
        return true;
    }

    // [MIX] Print a grid to either the terminal or a file.
    void printBoard(const char grid[SIZE][SIZE], std::ostream
& out) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                out << grid[r][c];
                if (c < SIZE - 1) {
```

```
                    out << ' ';
                }
            }
            out << '\n';
        }
    }

    // [GEMINI] [MIX] Recursive backtracking through all 81 c
ells.
    // [CHATGPT] Save each completed solution to an output fi
le.
    void search(int cell, std::ofstream& output) {
        if (cell == SIZE * SIZE) {
            ++solutionCount;

            // Display the solution in the terminal.
            std::cout << "\nSolution " << solutionCount <<
":\n";
            printBoard(board, std::cout);

            // Save the solution in the required file format.
            if (solutionCount > 1) {
                output << '\n';
            }
            printBoard(board, output);
            return;
        }

        int row = cell / SIZE;
        int col = cell % SIZE;

        if (board[row][col] != EMPTY) {
            search(cell + 1, output);
            return;
        }
```

```
        for (char digit = '1'; digit <= '9'; ++digit) {
            if (isSafe(row, col, digit)) {
                board[row][col] = digit;
                search(cell + 1, output);
                board[row][col] = EMPTY;
            }
        }
    }

public:
    SudokuSolver() : solutionCount(0) {}

    // [CHATGPT] Read and validate the puzzle file.
    bool loadPuzzle(const std::string& filename) {
        std::ifstream file(filename);

        if (!file.is_open()) {
            std::cerr << "Error: cannot open " << filename <<
'\n';
            return false;
        }

        std::string token;

        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (!(file >> token)) {
                    std::cerr << "Error: " << filename
                              << " has fewer than 81 cells
\n";
                    return false;
                }

                bool isBlank = (token == "_");
                bool isDigit = (token.length() == 1 &&
                                token[0] >= '1' && token[0] <
```

```
= '9');

                if (!isBlank && !isDigit) {
                    std::cerr << "Error: bad symbol '" << tok
en
                              << "' in " << filename << '\n';
                    return false;
                }

                board[r][c] = token[0];
                original[r][c] = token[0];
            }
        }

        return true;
    }

    // Print the original puzzle to the terminal.
    void printOriginal() const {
        printBoard(original, std::cout);
    }

    // [CHATGPT] Solve the puzzle and write results to a fil
e.
    void solve(const std::string& outputFilename) {
        std::ofstream output(outputFilename);

        if (!output.is_open()) {
            std::cerr << "Error: cannot create "
                      << outputFilename << '\n';
            return;
        }

        solutionCount = 0;
        search(0, output);
```

```
        if (solutionCount == 0) {
            std::cout << "No solution found\n";
            output << "No solution found.\n";
        } else {
            std::cout << "\nTotal solutions found: "
                      << solutionCount << '\n';

            output << "Solutions found: "
                   << solutionCount << '\n';
        }

        output.close();
    }
};

int main() {
    for (int i = 1; i <= NUM_PUZZLES; ++i) {
        std::string filename = "puzzle" + std::to_string(i) +
".txt";
        std::string outputFilename = "solution" + std::to_str
ing(i) + ".txt";

        std::cout << "========================================
=\n";
        std::cout << "File: " << filename << '\n';
        std::cout << "========================================
=\n";

        SudokuSolver solver;

        if (solver.loadPuzzle(filename)) {
            std::cout << "\nPuzzle:\n";
            solver.printOriginal();
            solver.solve(outputFilename);

            std::cout << "Output saved to: "
```

```
                    << outputFilename << '\n';
        }

        std::cout << '\n';
    }

    return 0;
}
```

ChatGPT — GenAI Code Analysis

a. GenAI program: ChatGPT (OpenAI).

b. Accessed through: The ChatGPT website (chatgpt.com).

c. Prompt: Modify the provided C++ Sudoku solver to save all solutions from puzzle1.txt through puzzle5.txt to solution1.txt through solution5.txt, using the same grid format and ending with “Solutions found: n” or “No solution found.”

d. Generated code: The C++ code that I pasted above

e. Correctness: The code passes all the tests and gives correct output.

Execution time: The code uses recursive backtracking and loops to check possible solutions, so execution time increases with the number of empty cells and possibilities explored.

Space complexity: O(1) space for the fixed-size puzzle

Maintainability: Descriptive function names, comments, and separate methods for loading, solving, and printing make the code easier to review and modify. Comments could be further expanded on for clarification and readability.

Claude:

```
#include <iostream>   // cout and cerr for status and error m
essages
#include <fstream>    // ifstream/ofstream for puzzle and sol
ution files
```

```
#include <string>     // string for file names and for readin
g symbols

const int SIZE = 9;          // the board is 9 x 9
const int BOX = 3;           // each block is 3 x 3
const char EMPTY = '_';      // symbol for a blank cell
const int NUM_PUZZLES = 5;   // puzzle1.txt through puzzle5.t
xt

class SudokuSolver {
private:
    char board[SIZE][SIZE];      // working grid that the sea
rch fills in and erases
    int solutionCount;           // number of full solutions
found so far
    std::ofstream out;           // solution file that soluti
ons are written to

    // [GEMINI] Checks whether 'digit' can go at (row, col).
One loop checks the
    // row, the column, and the 3x3 block at the same time.
    bool isSafe(int row, int col, char digit) const {
        int boxTop = BOX * (row / BOX);
        int boxLeft = BOX * (col / BOX);
        for (int i = 0; i < SIZE; ++i) {
            if (board[row][i] == digit) return false;
            if (board[i][col] == digit) return false;
            if (board[boxTop + i / BOX][boxLeft + i % BOX] ==
digit) return false;
        }
        return true;
    }

    // Writes the working board to the solution file in the s
ame format as the
    // puzzle files: one row per line, symbols separated by s
```

```
ingle spaces.
    void writeBoard() {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                out << board[r][c];
                if (c < SIZE - 1) {
                    out << ' ';
                }
            }
            out << '\n';
        }
    }

    // [GEMINI] [MIX] The recursive search over cells 0 to 8
0.
    // Base case: when cell reaches 81, the board is a comple
te solution. A
    // blank line is written first if this is not the first s
olution, so
    // solutions are separated, and then the board is written
to the file.
    // If the cell already holds a clue, move on to the next
cell. Otherwise
    // try digits 1-9, placing each safe one, recursing, and
erasing it again
    // (backtracking) so every solution is found, not just th
e first.
    void search(int cell) {
        if (cell == SIZE * SIZE) {
            if (solutionCount > 0) {
                out << '\n';
            }
            ++solutionCount;
            writeBoard();
            return;
        }
```

```
        int row = cell / SIZE;
        int col = cell % SIZE;
        if (board[row][col] != EMPTY) {
            search(cell + 1);
            return;
        }
        for (char digit = '1'; digit <= '9'; ++digit) {
            if (isSafe(row, col, digit)) {
                board[row][col] = digit;
                search(cell + 1);
                board[row][col] = EMPTY;
            }
        }
    }

public:
    SudokuSolver() : solutionCount(0) {}

    // [CHATGPT] Reads a puzzle file into the board, with inp
ut checking.
    // Returns false with an error message if the file will n
ot open, has
    // fewer than 81 symbols, or contains a symbol that is no
t "_" or 1-9.
    bool loadPuzzle(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error: cannot open " << filename <<
'\n';
            return false;
        }
        std::string token;
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (!(file >> token)) {
                    std::cerr << "Error: " << filename << " h
```

```
as fewer than 81 cells\n";
                    return false;
                }
                bool isBlank = (token == "_");
                bool isDigit = (token.length() == 1 && token
[0] >= '1' && token[0] <= '9');
                if (!isBlank && !isDigit) {
                    std::cerr << "Error: bad symbol '" << tok
en << "' in " << filename << '\n';
                    return false;
                }
                board[r][c] = token[0];
            }
        }
        return true;
    }

    // Opens the solution file, runs the search, and then wri
tes the final
    // line: "No solution found." if nothing was found, other
wise
    // "Solutions found: n". Returns the number of solutions,
or -1 if the
    // solution file could not be opened.
    int solve(const std::string& solutionFile) {
        out.open(solutionFile);
        if (!out.is_open()) {
            std::cerr << "Error: cannot write " << solutionFi
le << '\n';
            return -1;
        }
        solutionCount = 0;
        search(0);
        if (solutionCount == 0) {
            out << "No solution found.\n";
        } else {
```

```
            out << "\nSolutions found: " << solutionCount <<
'\n';
        }
        out.close();
        return solutionCount;
    }
};

// main goes through the five puzzle files. For each one it l
oads the puzzle,
// solves it, writes solutionN.txt, and prints a one-line sta
tus to the console.
int main() {
    for (int i = 1; i <= NUM_PUZZLES; ++i) {
        std::string puzzleFile = "puzzle" + std::to_string(i)
+ ".txt";
        std::string solutionFile = "solution" + std::to_strin
g(i) + ".txt";
        SudokuSolver solver;
        if (solver.loadPuzzle(puzzleFile)) {
            int count = solver.solve(solutionFile);
            if (count >= 0) {
                std::cout << puzzleFile << ": " << count
                          << " solution(s) written to " << so
lutionFile << '\n';
            }
        }
    }
    return 0;
}
```

Claude — GenAI Code Analysis

a. GenAI program: Claude

b. Accessed through: claude.ai

c. Prompt: same prompt as chatgpt

d. Generated code: The C++ code put above

e. Correctness: The code appears to produce the requested solution files and checks input errors

Execution time:O(9^n)

Space complexity: O(1) space

Maintainability: Clearly named functions and comments aid readability, but the file-output stream is tightly with the solver class so there is a possible change.

f. Selected GenAI Program

I chose Claude because its solution is simpler, uses only one Sudoku board, and avoids unnecessary console printing while still generating the required solution files

g. Improvements I Will Make

Correctness: I will make it so that the initial puzzle actually rejects duplicate digits in any row, column, or 3×3 box before solving

Execution time: I will skip consecutive pre-filled cells in a loop instead of making a separate recursive call for every given clue

I will pass the output stream by reference instead of storing it in the solver class. This uses less memory, while the overall space complexity stays O(1) for a fixed-size Sudoku puzzle.

Maintainability: I will make the board-printing function accept an output stream so it can be reused for console output or solution files